

1.

Question 1

Insert in the text box:

The essence of your solution to the challenge. [10 marks]

The Postfix application is like a calculator that inverts things. Instead of writing an arithmetic calculation as “3 + 4”, we write “3 4 +” in the Postfix application. This may sound like a small change, but it makes a big difference in the algorithms. The algorithm must scan an expression from left to right, using only a stack to store numbers. The algorithm will pop the two operands from the stack, then perform the operator and push the result back. Step by step, the stack will shrink until one number remains, which is the answer. The assignment wants us to make Postfix++, which supports variables. To implement this function, I built a symbol table to support the algorithm. It’s a small hash table, one slot for each of the capital letters A to Z, and each slot holds a short list to hold the values. This way, when more than one value is assigned to the key, it keeps the old value and adds the new one. I made the algorithm that when the algorithm finds a key assignment to the = sign like “A 5 =”, it assigns the key and value to the table. Later, if that letter is called out, the program will pull its value, which is 5 and perform the standard postfix arithmetic calculation.

10 points

2.

Question 2

Write in the text box:

A list of the **data structures** in your solution. Explain why each chosen data structure is suitable for the task. [20 marks]

Data structures come in two main types, linear and non-linear data structures. Each type has its use, depending on how it links and organises data. When using the postfix algorithm, the structure must keep the elements in order. It takes the first two numbers, then applies the operator. In postfix++, operands can be variables too. But the rule stays the same, which is to take the first two operands, then use the first operator. This makes the order of elements vital. That’s why linear data structures suit this task well. Linear structures keep elements in sequence by having one element follow another in a clear line (Coursera, 2024). Non-linear structures don’t follow one path. There’s no single “next” element. They use trees, heaps, tries, or graphs. These form levels or webs. Since one node can lead to many paths, we often use recursion or special algorithms to move through them (GeeksforGeeks, 2025).

Stacks

A stack is a linear data structure that enforces the Last-In-First-Out (LIFO) discipline. This means that the element inserted most recently is always the first one removed. All interaction happens at a single access point called the *top*. Adding an element, also known as a *push*, places it on the top, conceptually shoving earlier items one position down. Removing an element, also known as a *pop*, takes whichever item currently occupies the top and returns it. There are two other operations that support the stack data structure. The *peek* function inspects the top value without deleting it, while *isEmpty* simply reports whether the stack contains any elements. Since every operation touches only the top position, each of them runs in constant time $O(1)$ in time complexity.

In Postfix++, the stack is a better data structure to perform the operations on. Each number or variable is pushed to the stack when the user inputs it. When an operator shows up, the last two values entered are popped, the operator then performs it on the two values, and the result is pushed back to the stack. When there are other operands and operators, it will repeat until there are no more operands and operators. In a stack, all actions happen at the top, the interpreter runs in constant time for most of its functions at $O(1)$, and only uses memory based on how deeply the expression is nested, making it $O(n)$ in space. All of these features and benefits make the stack a smart, resource-friendly choice for the algorithm.

Before deciding on stacks, I did consider other data structures for this algorithm, but none of them work as well as stacks. The queue data structures follow a First-In, First-Out (FIFO) policy, which is the opposite of what we need. When an operator shows up, it will take too long to grab the last two items. The queue has to pop all the operands inside first to get the last element entered, then store them temporarily in another data structure, then get the last and second-last element and perform the operator on it. This could make the time and space complexity for the worst case to be $O(n^2)$.

Another data structure which I thought would work is an array or vector. However, the downsides of them are that removing items from the end often means shifting the rest. This increases the time complexity to $O(n)$, and adjusting the size may result in too much space being allocated, then we need to compensate for the additional values. This is because vectors have a fixed size and are not dynamic; this means that to include more values, I must copy everything and place it in another vector with more space, wasting time and space.

Hashtable

A hash table is a non-linear data structure that stores key-value pairs, but unlike arrays, it doesn't rely on order. Instead, it uses a hash function to turn each key into an index. That index points to where the value should go, making hashing fast. It also takes a key and runs it through a function that gives back a fixed-size result. That result is trimmed to fit the table and used as slots. The key's value is then placed in the slot. What makes hash tables powerful is their speed, where most operations take a time complexity of constant time of $O(1)$. Inserting, searching, or deleting a value usually takes constant time. That's why they're used in symbol tables, dictionaries, caches, and any system where fast access to data matters.

For the reason above, I use a hash table as a clean way to match names to values. A hash table works more like a dictionary, where each variable, A to Z, gets stored with its value in a fixed array. A hash function turns the variable name into an index, so I can find it fast. Both adding and looking up values usually take constant time $O(1)$, which I think is perfect for what Postfix++ needs.

The hash table has two main functions: the first is to remember the current value for each variable, and the second, to fetch that value right away when it pops up again. A hash table handles both. When the user types "=", the program inserts or updates the key-value pair in the table. Then, when the variable shows up later, the interpreter searches for it, grabs the value, and pushes it onto the stack. It all happens fast, without delay. Since the Postfix++ will be on a low-memory device, I think it makes sense to keep things simple. I map each capital letter to an index, giving me 26 clean slots, one for each letter. This hash table is also a better option, as there are no clashes, and every operation is truly $O(1)$ using direct addressing. The space needed is $O(n)$, where n is just the number of variables, and it doesn't grow beyond that.

Before deciding on the hash table, I considered using a binary search tree, which is another non-linear structure, but it proved to be a poor fit for Postfix++. Each variable lives in its node, linked to at most two others. To find or update a value, the interpreter must start at the root and work its way down. Even in a balanced tree, it takes around $\log n$ steps. If the tree becomes unbalanced, it can take up to n . That's much slower than the constant-time lookup I get with a simple hash table of 26 slots. The tree also comes with overhead. Every node needs pointers, and the code has to handle balancing. On a small device with limited memory, that's a waste. For something as lightweight as this calculator, I think a binary tree adds more complexity than value.

Linked List

The linked list data structure is a sequence of elements, each of which contains a reference to the next element. This enables the linked list to efficiently insert and delete elements. The linked list is also a dynamic data structure because its length can be easily changed. In comparison to arrays, a linked list data structure takes less time to perform operations such as insertion and deletion.

The reason for including a linked list in this algorithm is that the hash table needs a way to handle collisions. Collisions are tricky as if two variables hash to the same slot, one might overwrite the other. I think that's the core issue, as the moment one key wipes out another, the whole table starts giving wrong answers. To avoid that, I decided to link a small list to each of the twenty-six buckets in the symbol table. Each bucket contains a simple chain that grows only when needed. The first variable that lands in a bucket gets its node: it stores the key, the value, and a next pointer set to null. If a second key lands in the same spot, I just add a new node at the front.

The array still gives me a direct pointer in, and the list is usually only one or two steps long, resulting in a constant time complexity of $O(1)$. I feel this keeps the design light. Each bucket stores only a pointer, and every node needs just two words. The space grows only with the number of actual variables, not with the full size of the table. That's lean and efficient as it contains a linear time complexity of $O(n)$.

There are three main methods to handle collisions in a hash table. However, other methods don't fit Postfix++ as well. Extend-and-rehash means doubling the table and moving everything to the extended hash table. This is a costly and pointless approach here since the variable names are capped at A to Z. This means that there's nothing to extend, especially on a low-memory system. I can't justify all that extra copying. Linear probing stays in the array but runs into a mess called clustering. Once a few slots fill, each new probe stretches the line longer. That slows everything down. Deleting becomes awkward, too, as I would need "tombstones" to keep the probe chain working.

This leaves with the last method, which is separate chaining, and it's better implemented using a linked list. On average, everything still runs in constant time. Even in the worst-case scenario, every variable hashing to the same bucket results in a time complexity of $O(n)$, which is fine when n is no more than 26. Space is also reasonable, as each node adds a little, and each bucket holds one pointer. Compare that to other models; direct addressing consumes $O(1)$ cells, while open addressing often wastes a significant amount of space to remain efficient.

20 points

3.

Question 3

Write in the text box:

An explanation of the **original** algorithms of your solution in non-technical language. [10 marks]

To explain my algorithm for my solution in non-technical language, I would like to give an analogy to my solution. Picture the calculator's workspace like a child's play area. There's a notepad and a set of bright wooden blocks scattered across a small desk. Imagine the child as the user who would be using the Postfix++, and the blocks and paper table list as the data structures.

At the start, the child will first write all numbers and math symbols onto the blocks. For all the plain numbers, like 7, 12, 20, or even negative numbers like -7, -8, -9, the child will stack them up to a tower. Whichever number was written first by the child will be placed on the base, and the subsequent numbers will be placed on top of it, until there are no number blocks left. Right next to it, there is a second tower to represent the symbols received. This tower will stack from blocks with symbols like +, *, or =, the program writes it on the block and adds to that tower.

Once the two towers are formed, the child removes the first two blocks on top of the number tower and a single block from the symbol tower. Let's say if the symbol from the block taken out is *, it will multiply the two numbers block taken out from the integer tower. The result from the calculation will then be written on a new block and placed at the top of the integer tower. The child will repeat this until there are no more blocks left at the symbol tower, and there is only one last block in the integer tower. The remaining block will be the result of all the calculations done.

However, numbers are not the only element that can be placed on the integer tower. The child can also place letters like A or B. These letters represent variables, which are a key value pair.

The child can create a tower with a block “A” and a block “3”, and if the symbol block is “=”, the child will note the letter block as the key, and the number block as the value, and record it in the notepad.

With the updated notepad, whenever the child places the block “A” with another number block, for example “5”, the child can refer to the notepad to see what the value is for block “A”. In this case, since the notepad states that it’s “3”, then it will proceed to do the calculation “3 + 5”. Now and then, the same letter gets assigned more than once, in a scenario where the child first places block “A” and block “5” with the symbol “=”, followed by block “A” and block “8”, the notepad should be able to hold more than one value. Instead of deleting the 5, the child will just write the second value beside the first value, as the notepad is a table. This allows the child to choose either the first or the second value when performing calculations with the block “A” and a number block.

With just these two props, a pair of block towers and a simple notepad, the child can perform the Postfix++ algorithm. This analogy works the same way as my solution to create the Postfix++ algorithm, where the data structures are the block towers and notepad, and the child is the algorithm.

7.

Question 7

Point out any defects of your design and/or implementation. Suggest remedies for these shortcomings. [10 marks]

My Postfix++ keeps variables in a chained hash table, even though it only supports 26 uppercase letters. A defect in my algorithm solution might be that if we assume the user won’t use the full 26 letters as variables, we might waste a lot of resources. The hash function maps each letter to its slot, so there’s no real need for linked lists. The chains just add overhead: more pointer chasing, more heap traffic.

Every time a variable is reassigned, the interpreter creates a new node, walks the chain, and rewires the pointers. Each lookup from the linked list adds at least one extra indirection. On memory-tight, cache-sensitive devices, the cost shows up fast, as misses, fragmentation, and slowdown. The benefit of using a hash table is that direct addressing has an $O(1)$ time complexity when retrieving a value. However, with the linked list, reassigning the same letter a thousand times, what should be an $O(1)$ update becomes $O(n)$.

A better fit for the situation where the user won’t create 26 variables is open addressing with linear probing. We keep the same 26-slot array but store values directly in their home slots. Only when a rare collision happens do we probe forward. This works well when most programs only use a few letters. With a low load factor, inserts, lookups, and deletions all stay at $O(1)$ for time complexity, and rehashing is unlikely to occur when there are only 26 possible keys. However, if a script uses all 26 letters, linear probing is not the best option anymore. When the hash table

fills, using linear probing isn't a good option anymore, as deleting might leave tombstones. At that point, separate chaining becomes better again, as each bucket holds exactly one key, reassignments stay constant-time, and there's no probing at all.

The cleanest and most predictable option is to ensure there is direct addressing in the hash table. Set up a flat array of 26 cells, one per letter. Store the current value directly. No collisions, no chains, no probes. This can be done by ensuring that the hash table is simplistic, with a flat array of 26 cells, one per letter, and storing the current value directly. Replacing the value instead of handling the collision can also lead to making the hash table simpler. In a simple hash table, where every operation uses direct addressing, such as insert, search, and delete, the time complexity stays at $O(1)$. If I ever need version history, like the collision methods mentioned above, I can create a tiny stack to store only letters with multiple values.

Therefore, the best remedy for these shortcomings is to create two versions of the application. The first version is to use open addressing when the program touches fewer than 26 variables. If it goes beyond that, switch to the second version that handles these collisions with separate chaining or direct addressing. That way, I match the table to the actual usage, resulting in no wasted work, less memory, and Postfix++ stays fast, small, and easy to reason about.